1. General description

The Electric Vehicle Charging Station Locator enables EV drivers to search for charging stations in the vicinity and narrow down their results by filtering by various station details. The first concept was for a mobile app featuring an interactive map, the ability to locate the user's current position, and the ability to provide directions.

[REVISED] The project scope as required is a Python CLI. That's the only viable option given the project's timescale, and it allows the data retrieval and filtering logic to be tested in isolation. The current implementation uses the public ArcGIS Alternative Fueling Stations Feature Service. The app will take in a lat/long, query the electric fueling records in the vicinity, normalize the response, apply the filters, and then output the results and details. A small local JSON dataset is included as a dev/test fallback in the event that the live request fails.

2. Intended users and needs

Primary users are EV drivers looking for a charging station in the vicinity (including those planning longer journeys) and people who are still getting to grips with the various connector varieties and charging speeds. Their requirement is to be able to compare a reasonable number of charging options without having to visit multiple websites.

The V1 prototype can meet that requirement but is not a navigation or emergency-driving tool. The data may be partial or out of date, and the system cannot guarantee that a charger will be available in real time.

3. Required Version 1 features

• Search location: have the user enter the latitude and longitude.

[REVISED] City/address search and automatic device location are outside the scope of the requirement.

• Station retrieval: Get the electric charging station records from the ArcGIS Feature Service within a 25-mile radius, with a maximum of 100 records returned.

• Fallback data: If the live request fails, load data from data/sample_stations.json and be very clear to the user that this is the data that has been used.

• Data normalization: take the service response and turn it into a consistent internal structure with station name, address, coordinates, distance, connector info, power, price, status/access info, operator, number of chargers, hours, and pricing as appropriate.

• Distance calculation: calculate the approximate straight-line distance from the entered coordinates using the Haversine formula.

• Filters: Max distance, connector type, min charging power, max price per kWh, and an available only/status filter. Missing numeric values should be shown as unknown rather than made up.

• Results: Display matching stations sorted by distance, with the key attributes and clear labeling of the unknowns.

• Station details: Let the user pick a result and show the details available for that result.

• Repeat/exit: Allow the user to do another search or exit the program.

• Documentation: Maintain a README with setup/run instructions and a bug sheet for known issues

4. User activity flow

1. The user commences the program.

2. The user selects the search option.

3. The user enters the latitude and longitude.

4. The program requests station records from ArcGIS. If that fails, it loads the local sample dataset and reports that it has fallen back.

5. The program normalizes the records and calculates the distances where coordinates are available.

6. The user enters optional filter criteria.

7. The program displays the matching stations sorted by distance.

8. The user picks a station to view the details for or presses Enter to search again.

9. The user can commence another search or exit.

5. Data and technical approach

• Language: Python.

• Interface: Terminal/CLI.

• External data: ArcGIS Alternative Fueling Stations Feature Service query endpoint.

• Local test data: data/sample_stations.json

• Data format: JSON responses converted to Python lists and dictionaries.

• Network access: Python standard library URL/request utilities.

• Distance: Haversine calculation for straight-line distance.

[REVISED]

Pandas, Flask/Streamlit, and an interactive map library are all unnecessary for the core Version 1 implementation. Keeping dependencies as minimal as possible reduces the amount of setup and debugging required.

6. Known limitations

• The live data service is reliant on an internet connection and may be unavailable.

• The local sample dataset will be sufficient to test the workflow, yet of course does not represent a live search result.

• Price and charging power may be missing or difficult to extract for some records. If unknown, then the associated numeric filter will not be able to reliably include that station. This behavior should be tested and documented.

• The status/access fields are source-reported and should not be described as being a guarantee of real-time charger availability.

• The API query searches within a 25-mile radius and limits the response to 100 records. A user filter will not be able to show any stations which weren't returned by the first query.

• The current location input is just coordinates (no geocoding, no GPS, no routing, no directions).

• A successful live request and a complete local end-to-end run still need to be checked before you can say the entire workflow is reliable.

7. [NICE TO HAVE] features for later

• A simple web interface (using Flask or Streamlit)

• An interactive map view of the returned stations.

• City/address input or geocoding

• Links to external navigation instructions.

• Better interpretation of the various pricing and charging power formats.

• Automated tests and better handling of odd API responses.

These are optional and should not delay completion and testing of the CLI requirements

8. Completion criteria

1. The program starts and displays a working search/exit menu.

2. The user can enter valid coordinates and either get live results or a clearly identifiable sample data fallback.

3. Results can be filtered and sorted without crashing on missing optional fields.

4. The user can select a station and view the details, then search again or exit.

5. The README has the instructions for running the program and a list of known issues.

6. The core flow has been tested locally, and the review document correctly reports the outcome of those tests.